

CREACIÓN DE UNA RAMA EN GIT BASH PARA PROYECTO INTEGRADOR

CONTEXTO

Para el proyecto integrador se requiere crear una rama dentro del repositorio donde vendrá toda la documentación y presentación del mismo proyecto. A continuación se presentan detalladamente los pasos que se siguieron para la creación de la rama.

PARTE 1. CONCEPTOS BÁSICOS

1.¿Qué es un repositorio de git?

Un repositorio de GitHub es un espacio donde se almacena todo el código, los archivos y el historial de revisiones de un proyecto. Es el elemento más básico de GitHub y permite a los desarrolladores gestionar y colaborar en proyectos de software de manera eficiente.

2.¿Qué es una rama?

Las ramas en Git permiten trabajar en diferentes líneas de desarrollo dentro de un mismo proyecto, facilitando la colaboración y el manejo de cambios sin afectar el código principal.

3. ¿Qué es Git?

Sistema de control de versiones de distribución y gratuito, es un sistema que permite a los desarrolladores registrar cambios hechos en el código fuente de un programa. Algunas características principales son:

- Control de Distribución: Para cada persona tiene una copia completa del historial del proyecto en su propia computadora.
- Trabajo sin Conexión: Permite crear ramas, fusionar y revisar cambios sin estar conectado a internet

4. ¿Qué es GitHub?

Es una plataforma en nube que permite a los desarrolladores almacenar, administrar y compartir su código fuente usando el sistema de control de versiones Git. Funciona como punto de encuentro y red social para trabajo en equipo, permitiendo rastrear cambios, revisar errores y colaborar en proyectos de software desde cualquier lugar.

Sus funciones principales son:

- Repositorios: Almacenamiento de archivos e historiales de un proyecto, tanto públicos o privados.
- Control de versiones: Guarda modificaciones realizadas en el código para poder volver a un estado anterior si algo falla.
- Colaboración: Permite que varios programadores trabajen en el mismo proyecto a la vez sin sobrescribir el trabajo de los demás.

- Pull Requests: Solicitudes para proponer cambios o mejoras a un código antes de aplicar de forma definitiva.

5. ¿Cuál es la diferencia entre Git y GitHub?

Git: Es un programa que se instala en el computador para guardar y ver los cambios de los archivos.

GitHub: Es una página web en internet que guarda las copias de los proyectos para que se pueda trabajar con otras personas.

6. ¿Qué es un repositorio?

Es una carpeta especial de un proyecto que guarda todos los archivos, textos e historial completo de cambios. Git: es el programa que controla los cambios en la computadora. GitHub: Sitio web en nube donde guarda y comparte estos repositorios para el trabajo en conjunto con otras personas.

7. ¿Qué es un commit?

Es una instantánea o "fotografía" del estado de tu proyecto en un momento determinado dentro de un sistema de control de versiones como Git. Guarda los cambios realizados en los archivos, junto con un mensaje explicativo, un identificador único (hash) y el autor de la modificación.

8. ¿Para qué sirve un archivo README.md?

Es la carta de presentación y guía principal de un repositorio. Escrito en formato Markdown (`.md`), sirve para explicar de qué trata el proyecto y cómo usarlo.

- Descripción del proyecto: Explica qué hace la aplicación o biblioteca y cuál es su propósito.
- Instrucciones de instalación: Detalla los pasos y dependencias necesarios para ejecutar el proyecto de manera local.
- Guía de uso: Proporciona ejemplos de código, comandos o capturas de pantalla para mostrar cómo funciona.
- Documentación y contribución: Incluye reglas para colaborar, licencias de uso y créditos a los creadores.

PARTE 2. VERIFICACIÓN DE INSTALACIÓN

Paso 1. Verificar que nuestro git bash funcione mediante git --versión.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~  
$ git --version  
git version 2.52.0.windows.1
```

Paso 2. Posterior a la verificación de versión, debes configurar tu nombre y tu correo, en nuestro caso ya existe.

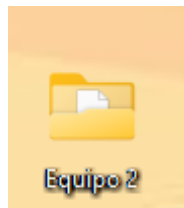
```
kevin@LAPTOP-F02EJHTV MINGW64 ~
```

Paso 3. Comprueba tu configuración y si está correcta, es que todo salió bien.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~  
$ git config --global --list  
user.name=keyvler04  
user.email=keyvler04@gmail.com
```

PARTE 3. CREAR REPOSITORIO LOCAL

Paso 1. Crea una carpeta en tu computadora llamada: Equipo-2.



Paso 2. Entra a la carpeta desde la terminal de bash y coloca los siguientes comandos para inicializar git:

```
damar@Danie23 MINGW64 ~/OneDrive/Escritorio/Equipo 2  
$ git init
```

Paso 3. Crea un archivo README.md con los siguientes comandos:

```
damar@Danie23 MINGW64 ~/OneDrive/Escritorio/Equipo 2
$ echo "# EQUIPO-2" >> README.md
```

Paso 4. Revisa el estado:

```
damar@Danie23 MINGW64 ~/OneDrive/Escritorio/Equipo 2 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.
```

Paso 5. Crea tu primer commit:

```
damar@Danie23 MINGW64 ~/OneDrive/Escritorio/Equipo 2
$ git commit -m "repositorio"
```

Paso 6. Revisa el historial:

```
damar@Danie23 MINGW64 ~/OneDrive/Escritorio/Equipo 2 (main)
$ git log --oneline
2c3d605 (HEAD -> main, origin/main) first commit
fab3c0c first commit
```

Rama y su implementación de documentos

Paso 1. Se debe clonar el repositorio de la persona administradora

```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1
$ git clone "https://github.com/danierdz55-commits/EQUIPO-2-.git"
Cloning into 'EQUIPO-2-'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Paso 2. Verifica estar dentro.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1
$ ls
EQUIPO-2-/
```

Paso 3. Entra en el repositorio.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1
$ cd EQUIPO-2-
```

Paso 4. Genera la rama y verificarla.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1/EQUIPO-2- (main)
$ git branch tarea1

kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1/EQUIPO-2- (main)
$ git checkout tarea1
Switched to branch 'tarea1'
```

Paso 5. Crea carpeta para ingresar documentos e ingresa a esa carpeta.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1/EQUIPO-2- (tarea1)
$ mkdir tarea1

kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1/EQUIPO-2- (tarea1)
$ cd tarea1
```

Nota: Podemos visualizar que estamos dentro de la rama por que cambia directo el nombre de main a el nombre asignado.

```
(main) (tarea1)
```

Paso 6. Verifica los documentos.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1/EQUIPO-2-/tarea1 (tarea1)
$ ls
'EQUIPO 2 (1).pptx'
'TAREA 1_ Definición de Perfil Profesional y Enfoque de Presentación (EQUIPO 2 ).docx'
```

Paso 7. Agrega un comentario antes de subir los archivos, visible para todos.

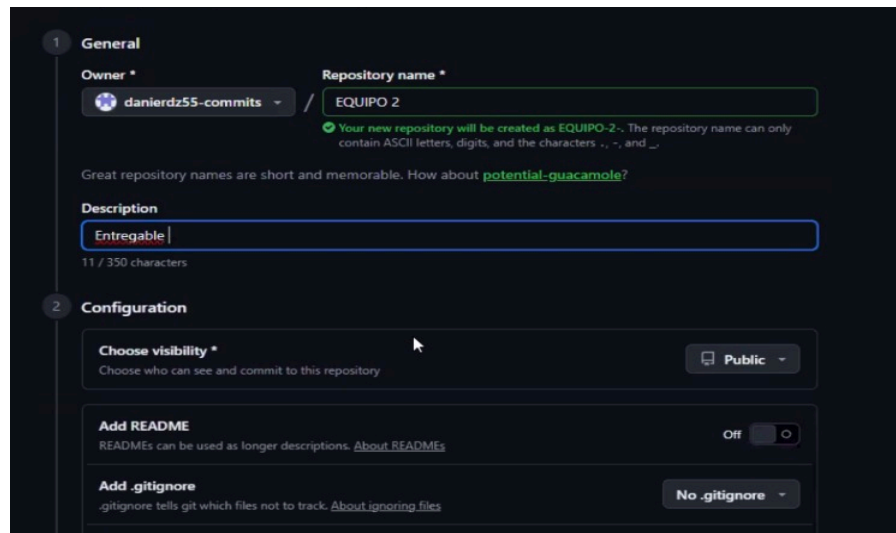
```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1/EQUIPO-2-/tarea1 (tarea1)
$ git commit -m "añadimos tarea del equipo 2 y su rama"
[tarea1 90ab2b1] añadimos tarea del equipo 2 y su rama
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 tarea1/EQUIPO 2 (1).pptx
create mode 100644 "tarea1/TAREA 1_ Definición de Perfil Profesional y Enfoque de Presentación (EQUIPO 2 ).docx"
```

Paso 8. Por último, debes subir los cambios al repositorio para que se guarden.

```
kevin@LAPTOP-F02EJHTV MINGW64 ~/Desktop/tarea_1/EQUIPO-2-/tarea1 (tarea1)
$ git push origin tarea1
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 504.28 KiB | 14.01 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'tarea1' on GitHub by visiting:
remote:   https://github.com/danierdz55-commits/EQUIPO-2-/pull/new/tarea1
remote:
```

PARTE 4. SUBIR EL REPOSITORIO A GitHub

Paso 1: Entrar a GitHub y crear un repositorio nuevo llamado: EQUIPO-2-.



The screenshot shows the GitHub repository creation interface. The 'General' tab is active, showing the repository name 'EQUIPO 2' and a description 'Entregable'. The 'Configuration' tab is also visible, showing options for visibility (Public) and adding a README or .gitignore file.

1 General

Owner * danierdz55-commits / Repository name * EQUIPO 2

✓ Your new repository will be created as EQUIPO-2-. The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

Great repository names are short and memorable. How about [potential-guacamole?](#)

Description Entregable

11 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

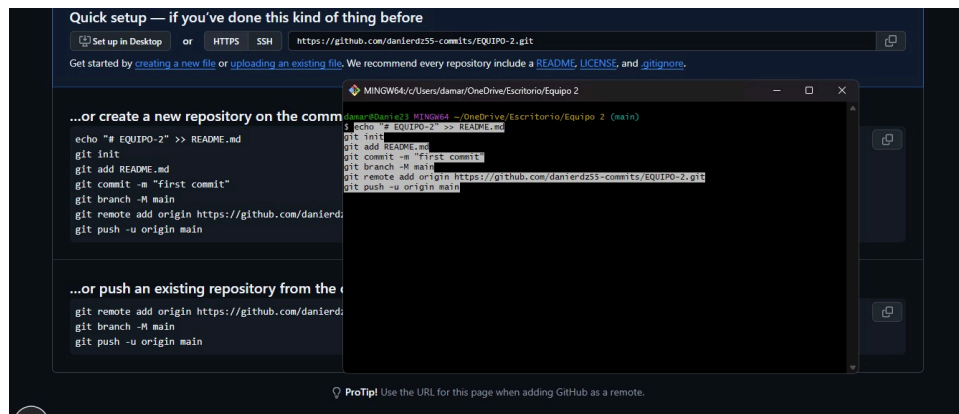
Add README Off

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Paso 2: Se copia la URL del repositorio. Después se abre la consola Bash desde la carpeta local EQUIPO-2-, se pega el código del repositorio y se ejecuta:



The screenshot shows a terminal window with the following commands and output:

```
Quick setup — if you've done this kind of thing before
Set up in Desktop or HTTPS SSH https://github.com/danierdz55-commits/EQUIPO-2.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.

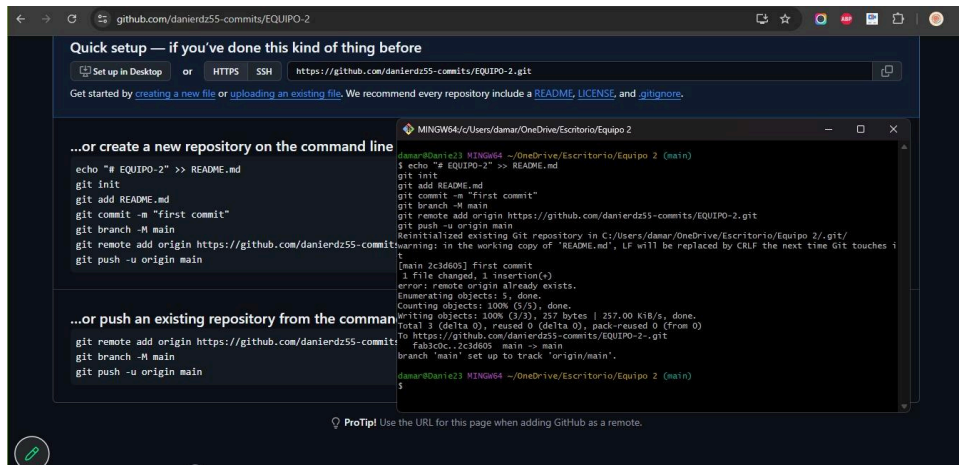
...or create a new repository on the command line
echo "# EQUIPO-2" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/danierdz55-commits/EQUIPO-2.git
git push -u origin main

...or push an existing repository from the command line
git remote add origin https://github.com/danierdz55-commits/EQUIPO-2.git
git branch -M main
git push -u origin main
```

MINGW64/c/Users/damar/OneDrive/Escritorio/Equipo 2 (main)

```
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/danierdz55-commits/EQUIPO-2.git
git push -u origin main
```

ProTip! Use the URL for this page when adding GitHub as a remote.

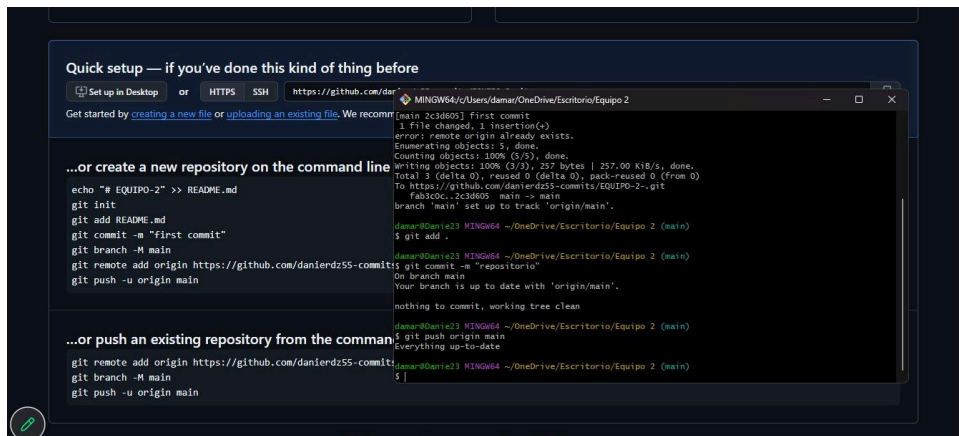


Paso 3: Ingresar los comandos en la consola bash:

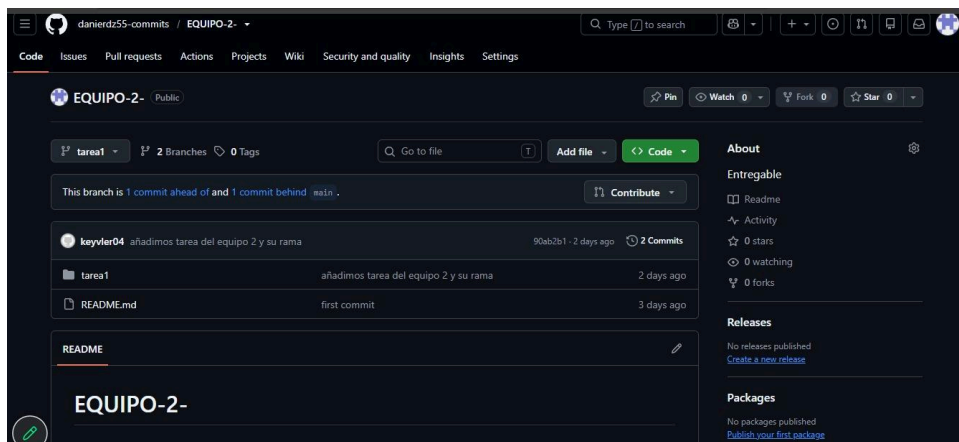
git add .

git commit -m "repositorio"

git push origin main



Paso 4: Verificamos en GitHub que aparezca el repositorio EQUIPO-2-



PARTE 5. REFLEXIÓN FINAL

1. ¿Qué parte del proceso se te hizo más fácil?
 - Crear el repositorio y reunir la información sobre las vacantes.
2. ¿Qué error o dificultad tuviste?
 - Documentar correctamente el proceso. La rama del repositorio. Agregar los documentos a la rama.
3. ¿Cómo resolverías un problema si git push no funciona?
 - Volver a comprobar los comandos o subir desde el escritorio los archivos directo a GitHub.
4. ¿Cómo podrías usar GitHub como portafolio profesional?
 - Agregando archivos con documentos y presentaciones para mostrar.
5. ¿Cómo se relaciona Git con el soporte TI?
 - Para colaborar en equipo de manera simultánea. Complementar información y resolver incidencias todos juntos desde un repositorio.